Cletus: Section 1, Day 2

- Date: [YYYY-MM-DD]
- Course: Week 1 Day 2, the building blocks of LLM engineering, frontier vs open-source models, and the Chat Completions API
- Lab notebook: week1/day2.ipynb (solution in week1/solution.py)
- Time spent: [e.g. 2h]

Key concepts

- The course is built around three things: models (closed frontier ones and open-source ones), tools (Ollama, Hugging Face, Gradio and so on) and techniques (APIs, prompting, RAG, fine-tuning, agents).
- Frontier models: GPT from OpenAI, Claude from Anthropic, Gemini from Google and Grok from xAI. They're the strongest, and you pay per token.
- Open-source (open-weight) models: Llama from Meta, Mistral, DeepSeek, Qwen and Gemma. Free to download, and the small ones run locally through Ollama.
- Distillation: training a small model to copy a bigger one. `deepseek-r1:1.5b` is DeepSeek R1 distilled into a small Qwen model.
- Chat Completions API: you give it a conversation and it predicts the next message. OpenAI came up with it and now almost every provider supports it.
- Endpoint: a URL that we send an HTTP request to. The `openai` Python package is only a small wrapper around that request. It doesn't contain any model.
- OpenAI-compatible endpoints: Gemini, Ollama and others accept the same request format, so the same `OpenAI` client works for all of them. Only `base_url` and `api_key` change.

Code / examples

The same call done two ways:

```
import os, requests

# 1) Raw HTTP request to the endpoint
headers = {"Authorization": f"Bearer {os.getenv('OPENAI_API_KEY')}", "Content-Type":
"application/json"}
payload = {"model": "gpt-5-nano", "messages": [{"role": "user", "content": "Tell me a fun fact"}]}
r = requests.post("https://api.openai.com/v1/chat/completions", headers=headers, json=payload)
print(r.json()["choices"][0]["message"]["content"])

# 2) Same thing with the client library
from openai import OpenAI
openai = OpenAI()
response = openai.chat.completions.create(model="gpt-5-nano", messages=payload["messages"])
print(response.choices[0].message.content)
```

To switch provider, only the base_url and the key change:

```
gemini = OpenAI(base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
                api_key=os.getenv("GOOGLE_API_KEY"))
ollama = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")  # any text works locally

ollama.chat.completions.create(model="llama3.2", messages=[{"role": "user", "content": "Tell me a
fun fact"}])
```

To check Ollama is up, open http://localhost:11434. It should say "Ollama is running". If it doesn't, run `ollama serve`. Models are downloaded with `ollama pull llama3.2`.

Exercise / challenge

Homework: change the Day 1 summarizer to use Llama 3.2 through Ollama instead of OpenAI. Only two things change, the client (base_url) and the model name. - Compare Llama's summary with GPT's for the same site. - Running locally costs nothing and the data stays on your machine, but the model is much weaker than the frontier ones.

Questions & blockers

Things to watch out for: - `llama3.3` and `llama4` are too big for a normal laptop. Stick with `llama3.2`, or `llama3.2:1b` if it's slow. - After editing `.env`, save it and reload with `load_dotenv(override=True)`. - Gemini keys start with `AIz` (or `AQ.`). You can get one free at https://aistudio.google.com/api-keys.

To discuss with my partner

- When would we pick a local open-source model over a frontier API (cost, privacy, quality)?
- How different were the Llama and GPT summaries?

Resources

- Endpoints and HTTP basics: guides/04_technical_foundations.ipynb
- Code for Ollama, Gemini and OpenRouter: guides/09_ai_apis_and_ollama.ipynb
- Ollama: https://ollama.com